



哈爾濱工業大學 (深圳)
HARBIN INSTITUTE OF TECHNOLOGY

实验报告

开课学期: 2024 秋季

课程名称: 操作系统

实验名称: 基于 FUSE 的青春版 EXT2 文件系统

学生班级: 5

学生学号: 220110515

学生姓名: 金正达

评阅教师: _____

报告成绩: _____

实验与创新实践教育中心制

2024 年 9 月

一、实验详细设计

图文并茂地描述实验实现的所有功能和详细的设计方案及实验过程中的特色部分。

1、 总体设计方案

详细阐述文件系统的总体设计思路，包括系统架构图和关键组件的说明。

文件系统结构如下：



该简易 ext2 文件系统布局为：
超级块 1 逻辑块，索引节点位图 1 逻辑块，数据位图 1 逻辑块，索引节点 12 逻辑块，数据区 4081 逻辑块。
一个逻辑块为两个 IO 块的大小，即 1024B。
在此基础上，实现了文件系统的挂载、卸载、创建文件和目录、显示文件和目录的功能。

2、 功能详细说明

每个功能点的详细说明（关键的数据结构、核心代码、流程等）

1. 挂载

```
/**
 * @brief 挂载（mount）文件系统
 *
 * @param conn_info 可忽略，一些建立连接相关的信息
 * @return void*
 */
void* newfs_init(struct fuse_conn_info * conn_info) {
    /* TODO: 在这里进行挂载 */
    newfs_mount(newfs_options);
    return NULL;
}
```

挂载过程直接调用了辅助函数进行挂载。
辅助函数中，首先，
获取基本的 io 信息，并创建新的根目录，读取存储在磁盘中的超级块：

```
int newfs_mount(struct custom_options options) {
    super.fd = ddriver_open(options.device);

    int sz_disk;
    ddriver_ioctl(super.fd, IOC_REQ_DEVICE_SIZE, &sz_disk);
    ddriver_ioctl(super.fd, IOC_REQ_DEVICE_IO_SZ, &super.sz_io);
    super.sz_blks = super.sz_io * 2;
    super.blks_nums = sz_disk / super.sz_blks;
    super.ino_max = 614;

    struct newfs_dentry* root_dentry;
    root_dentry = new_dentry("/", NEWFS_TYPE_DIR);

    struct newfs_super_d super_d;
    newfs_driver_read(0, &super_d, sizeof(struct newfs_super_d));
}
```

接着，判断文件系统是否已经初始化，若没有初始化，则填充超级块、索引位图块、数据位图块、索引节点块以及数据块的位置和大小，并标记进行了初始化：

```

int is_init = 0;
if (super_d.magic != NEWFS_MAGIC) {
    super_d.sz_usage = 0;

    super_d.sb_blks = 1;
    super_d.sb_offset = 0;

    super_d.ino_map_blks = 1;
    super_d.ino_map_offset = super_d.sb_offset + super_d.sb_blks * super.sz_blks;

    super_d.data_map_blks = 1;
    super_d.data_map_offset = super_d.ino_map_offset + super_d.ino_map_blks * super.sz_blks;

    super_d.ino_blks = 12;
    super_d.ino_offset = super_d.data_map_offset + super_d.data_map_blks * super.sz_blks;

    super_d.data_blks = super.blks_nums - 16;
    super_d.data_offset = super_d.ino_offset + super_d.ino_blks * super.sz_blks;

    is_init = 1;
}

```

然后，将磁盘超级块中的信息填入内存超级块中，并创建读取索引和数据位图：

```

super.sz_usage = super_d.sz_usage;
super.sb_offset = super_d.sb_offset;
super.sb_blks = super_d.sb_blks;
super.ino_offset = super_d.ino_offset;
super.ino_blks = super_d.ino_blks;
super.ino_map_offset = super_d.ino_map_offset;
super.ino_map_blks = super_d.ino_map_blks;
super.data_map_offset = super_d.data_map_offset;
super.data_map_blks = super_d.data_map_blks;
super.data_offset = super_d.data_offset;
super.data_blks = super_d.data_blks;

super.inode_map = (uint8_t *)malloc(super.ino_map_blks * super.sz_blks);
super.data_map = (uint8_t *)malloc(super.data_map_blks * super.sz_blks);

newfs_driver_read(super.ino_map_offset, super.inode_map, super.ino_map_blks * super.sz_blks);
newfs_driver_read(super.data_map_offset, super.data_map, super.data_map_blks * super.sz_blks);

```

最后，如果此次挂载进行了初始化，就为根目录分配新的 inode，否则，读取磁盘中的文件的层级结构，完成挂载：

```

struct newfs_inode* root_inode;
if (is_init == 1) {
    root_inode = newfs_alloc_inode(root_dentry);
    root_inode = newfs_read_inode(root_dentry, 0);
    root_dentry->inode = root_inode;
} else {
    newfs_read_inode_remount(root_dentry, 0);
}
super.root_dentry = root_dentry;

return 0;

```

其中，new_dentry 函数用于创建并初始化一个新的目录项。该函数接收目录项的名称和文件类型作为参数，为目录项分配内存并将其初始化为零，name、ftype、dno、ino、inode、parent 和 brother 字段被初始化：

```

static inline struct newfs_dentry* new_dentry(char * name, NEWFS_FILE_TYPE ftype) {
    struct newfs_dentry * dentry = (struct newfs_dentry *)malloc(sizeof(struct newfs_dentry));
    memset(dentry, 0, sizeof(struct newfs_dentry));
    memcpy(dentry->name, name, MAX_NAME_LEN);
    dentry->ftype = ftype;
    dentry->dno = 0;
    dentry->ino = 0;
    dentry->inode = NULL;
    dentry->parent = NULL;
    dentry->brother = NULL;
    return dentry;
}

```

其中，newfs_driver_read 函数用于从磁盘中指定偏移处读取指定大小的文件输出。该函数根据块大小对输入的偏移量进行向下和向上对齐，计算出需要读取的块数，再通过文件描述符读取相应的数据块，并将读取的内容拷贝到指定的输出缓冲区：

```

int newfs_driver_read(int offset, void *out_content, int size) {
    int offset_aligned_down = NEWFS_ROUND_DOWN(offset, super.sz_io);
    int bias = offset - offset_aligned_down;
    int offset_aligned_up = NEWFS_ROUND_UP(offset_aligned_down + size + bias, super.sz_io);

    int blk_num = (offset_aligned_up - offset_aligned_down) / super.sz_io;
    char *buf = (char *)malloc(super.sz_io * blk_num);

    ddriver_seek(super.fd, offset_aligned_down, SEEK_SET);
    for (int i = 0; i < blk_num; i++) {
        ddriver_read(super.fd, buf + i * super.sz_io, super.sz_io);
    }
    memcpy(out_content, buf + bias, size);
    free(buf);
    return size;
}

```

其中，newfs_alloc_inode 函数用于分配并初始化一个新的 inode。该函数首先通过遍历索引节点位图查找一个空闲的 inode，并设置其对应的位为已使用：

```

struct newfs_inode* newfs_alloc_inode(struct newfs_dentry *dentry) {
    struct newfs_inode* inode;
    int byte_cursor = 0;
    int bit_cursor = 0;
    int ino_cursor = 0;
    int is_find_free = 0;

    for (byte_cursor = 0; byte_cursor < super.ino_map_blks * super.sz_blks; byte_cursor++)
    {
        for (bit_cursor = 0; bit_cursor < 8; bit_cursor++) {
            if((super.inode_map[byte_cursor] & (0x1 << bit_cursor)) == 0) {
                super.inode_map[byte_cursor] |= (0x1 << bit_cursor);
                is_find_free = 1;
                break;
            }
            ino_cursor++;
        }
        if (is_find_free) {
            break;
        }
    }

    if (!is_find_free || ino_cursor == super.ino_max) {
        return NULL;
    }
}

```

找到空闲 inode 后，函数为该 inode 分配内存，设置其基本属性，并将其与传入的 dentry 关联，dentry 指向新分配的 inode，而 inode 也指回对应的 dentry。如果 inode 类型为文件，为该文件分配数据存储空间，最后返回指向 inode 的指针：

```

inode = (struct newfs_inode*)malloc(sizeof(struct newfs_inode));
inode->ino = ino_cursor;
inode->size = 0;

/* dentry指向inode */
dentry->inode = inode;
dentry->ino = inode->ino;

/* inode指回dentry */
inode->dentry = dentry;

inode->dir_cnt = 0;
inode->dentrys = NULL;

if (inode->dentry->ftype == NEWFS_TYPE_FILE) {
    inode->data = (int *)malloc(NEWFS_DATA_PER_FILE * super.sz_blks);
}

return inode;

```

其中，newfs_read_inode 函数用于从存储介质中读取指定 inode 的数据，并初始化一个新的 inode 结构体。函数首先从指定的 inode 位置读取其基本信息，并设置到新分配的 inode 结构体中：

```

struct newfs_inode* newfs_read_inode(struct newfs_dentry * dentry, int ino) {
    struct newfs_inode* inode = (struct newfs_inode*)malloc(sizeof(struct newfs_inode));
    struct newfs_inode_d inode_d;
    struct newfs_dentry* sub_dentry;
    struct newfs_dentry_d dentry_d;
    int dir_cnt = 0, i, offset;

    offset = super.ino_offset + ino * sizeof(struct newfs_inode_d);
    newfs_driver_read(offset, &inode_d, sizeof(struct newfs_inode_d));
    inode->dir_cnt = 0;
    inode->ino = inode_d.ino;
    inode->dno = inode_d.dno;
    inode->size = inode_d.size;
    inode->dentry = dentry;
    inode->dentrys = NULL;

```

如果该 inode 对应的是目录类型，则函数继续读取该目录下的所有子目录项，并为每个子目录项创建新的 dentry 结构体，将其链接到当前 inode 的 dentrys 列表中：

```

if (inode->dentry->ftype == NEWFS_TYPE_DIR) {
    dir_cnt = inode_d.dir_cnt;
    offset = super.data_offset + dentry->dno * super.sz_blks;
    for (i = 0; i < dir_cnt; i++) {
        newfs_driver_read(offset + i * sizeof(struct newfs_dentry_d), &dentry_d, sizeof(struct newfs_dentry_d));
        sub_dentry = new_dentry(dentry_d.name, dentry_d.ftype);
        sub_dentry->parent = inode->dentry;
        sub_dentry->ino = dentry_d.ino;
        sub_dentry->dno = dentry_d.dno;
        if (inode->dentrys == NULL) {
            inode->dentrys = sub_dentry;
        } else {
            sub_dentry->brother = inode->dentrys;
            inode->dentrys = sub_dentry;
        }
        inode->dir_cnt++;
    }
} else if (inode->dentry->ftype == NEWFS_TYPE_FILE) {
}
return inode;

```

其中，newfs_read_inode_remount 函数用于未进行初始化时读取磁盘中的文件层次，基本结构与 newfs_read_inode 函数一致，只是增加了递归读取目录的过程调用：

```

if (inode->dentry->ftype == NEWFS_TYPE_DIR) {
    dir_cnt = inode_d.dir_cnt;
    offset = super.data_offset + inode->dno * super.sz_blks;
    for (i = 0; i < dir_cnt; i++) {
        int read_offset = offset + i * sizeof(struct newfs_dentry_d);
        newfs_driver_read(read_offset, &dentry_d, sizeof(struct newfs_dentry_d));
        sub_dentry = new_dentry(dentry_d.name, dentry_d.ftype);
        sub_dentry->parent = inode->dentry;
        sub_dentry->ino = dentry_d.ino;
        sub_dentry->dno = dentry_d.dno;
        if (inode->dentrys == NULL) {
            inode->dentrys = sub_dentry;
        } else {
            sub_dentry->brother = inode->dentrys;
            inode->dentrys = sub_dentry;
        }
        inode->dir_cnt++;
        newfs_read_inode_remount(sub_dentry, sub_dentry->ino);
    }
} else if (inode->dentry->ftype == NEWFS_TYPE_FILE) {
}
dentry->inode = inode;

```

2. 卸载

```

/**
 * @brief 卸载 (umount) 文件系统
 *
 * @param p 可忽略
 * @return void
 */
void newfs_destroy(void* p) {
    /* TODO: 在这里进行卸载 */
    newfs_umount();
    return;
}

```

卸载过程调用辅助函数 newfs_umount：

辅助函数中，首先从根节点开始向下将内存中的数据写回磁盘：

```

    struct newfs_super_d super_d;

    newfs_sync_inode(super.root_dentry->inode);

```

再填充写回超级块的信息:

```

super_d.magic      = NEWFS_MAGIC;
super_d.sz_io      = super.sz_io;
super_d.sz_blks    = super.sz_blks;
super_d.blks_nums  = super.blks_nums;
super_d.sz_usage   = super.sz_usage;
super_d.sb_offset  = super.sb_offset;
super_d.sb_blks    = super.sb_blks;
super_d.ino_offset = super.ino_offset;
super_d.ino_blks   = super.ino_blks;
super_d.ino_map_offset = super.ino_map_offset;
super_d.ino_map_blks = super.ino_map_blks;
super_d.data_map_offset = super.data_map_offset;
super_d.data_map_blks = super.data_map_blks;
super_d.data_offset = super.data_offset;
super_d.data_blks  = super.data_blks;

```

最后，写回超级块、索引位图和数据位图，释放位图变量的空间，关闭设备:

```

newfs_driver_write(0, &super_d, sizeof(struct newfs_super_d));
newfs_driver_write(super_d.ino_map_offset, super.ino_map, super.ino_map_blks * super.sz_blks);
newfs_driver_write(super_d.data_map_offset, super.data_map, super.data_map_blks * super.sz_blks);

free(super.inode_map);
free(super.data_map);
ddriver_close(super.fd);

return 0;

```

其中，`newfs_sync_inode` 函数用于同步 `inode` 及其相关的目录项数据到存储介质。函数先将 `inode` 的基本信息写入磁盘:

```

int newfs_sync_inode(struct newfs_inode * inode) {
    struct newfs_inode_d inode_d;
    struct newfs_dentry* dentry_cursor;
    struct newfs_dentry_d dentry_d;
    inode_d.ino = inode->ino;
    inode_d.dno = inode->dno;
    inode_d.size = inode->size;
    inode_d.dir_cnt = inode->dir_cnt;
    int offset;
    newfs_driver_write(super.ino_offset + inode->ino * sizeof(struct newfs_inode_d), &inode_d, sizeof(struct newfs_inode_d));
}

```

如果该 `inode` 对应的是一个目录，函数会遍历目录中的所有目录项，并将每个目录项的详细信息写入磁盘。如果某个目录项对应的 `inode` 不为空，函数会递归调用自己来同步该 `inode`:

```

if (inode->dentry->ftype == NEWFS_TYPE_DIR) {
    dentry_cursor = inode->dentry;
    if (dentry_cursor != NULL) {
        offset = super.data_offset + dentry_cursor->dno * super.sz_blks;
    }
    while (dentry_cursor != NULL)
    {
        memcpy(dentry_d.name, dentry_cursor->name, MAX_NAME_LEN);
        dentry_d.ftype = dentry_cursor->ftype;
        dentry_d.ino = dentry_cursor->ino;
        dentry_d.dno = dentry_cursor->dno;
        newfs_driver_write(offset, &dentry_d, sizeof(struct newfs_dentry_d));

        if (dentry_cursor->inode != NULL) {
            newfs_sync_inode(dentry_cursor->inode);
        }

        dentry_cursor = dentry_cursor->brother;
        offset += sizeof(struct newfs_dentry_d);
    }
} else if (inode->dentry->ftype == NEWFS_TYPE_FILE) {
}
return 0;

```

其中，`newfs_driver_write` 函数用于将数据写入存储介质，并确保数据按照块对齐方式存储。该函数首先计算并对齐偏移量，然后读取目标位置的原数据，将传入的数据复制到

缓冲区的相应位置，再将更新后的数据按块写回存储介质：

```
int newfs_driver_write(int offset, void *in_content, int size) {
    int offset_aligned_down = NEWFS_ROUND_DOWN(offset, super.sz_io);
    int bias = offset - offset_aligned_down;
    int offset_aligned_up = NEWFS_ROUND_UP(offset_aligned_down + size + bias, super.sz_io);

    int blk_num = (offset_aligned_up - offset_aligned_down) / super.sz_io;
    char *buf = (char *)malloc(super.sz_io * blk_num);
    newfs_driver_read(offset_aligned_down, buf, super.sz_io * blk_num);
    memcpy(buf + bias, in_content, size);

    ddriver_seek(super.fd, offset_aligned_down, SEEK_SET);
    for (int i = 0; i < blk_num; i++) {
        ddriver_write(super.fd, buf + i * super.sz_io, super.sz_io);
    }
    free(buf);
    return size;
}
```

3. 创建目录

`newfs_mkdir` 函数使用 `newfs_lookup` 查找路径中的最后一个目录项，如果该目录项已存在或者最后一个目录项对应的是一个文件而非目录，则返回错误码：

```
/**
 * @brief 创建目录
 *
 * @param path 相对于挂载点的路径
 * @param mode 创建模式（只读？只写？），可忽略
 * @return int 0成功，否则返回对应错误号
 */
int newfs_mkdir(const char* path, mode_t mode) {
    /* TODO: 解析路径，创建目录 */
    (void)mode;
    int is_find, is_root;
    char* name;
    struct newfs_dentry* last_dentry = newfs_lookup(path, &is_find, &is_root);
    struct newfs_dentry* dentry;
    struct newfs_inode* inode;

    if (is_find) {
        return -17;
    }

    if (last_dentry->inode->dentry->ftype == NEWFS_TYPE_FILE) {
        return -6;
    }
}
```

之后，函数从路径中提取目录名称，并创建一个新的目录项，调用 `newfs_alloc_inode` 为新目录分配一个 `inode`，并通过 `newfs_alloc_dentry` 将该目录项添加到父目录的目录项列表中：

```
name = newfs_get_fname(path);
dentry = new_dentry(name, NEWFS_TYPE_DIR);
dentry->parent = last_dentry;
inode = newfs_alloc_inode(dentry);
newfs_alloc_dentry(last_dentry->inode, dentry);
return 0;
```

其中，`newfs_lookup` 函数用于根据路径查找文件系统中的目录项。函数首先计算路径的层级数，并复制路径字符串。如果路径层级为 0，表示查询的是根目录，直接返回根目录项并标记为找到：

```

struct newfs_dentry* newfs_lookup(const char * path, int* is_find, int* is_root) {
    struct newfs_dentry* dentry_cursor = super.root_dentry;
    struct newfs_dentry* dentry_ret = NULL;
    struct newfs_inode* inode;
    int total_lvl = newfs_calc_lvl(path);
    int lvl = 0;
    int is_hit;
    char* name = NULL;
    char* path_cpy = (char*)malloc(sizeof(path));
    *is_root = 0;
    strcpy(path_cpy, path);

    if (total_lvl == 0) { /* 根目录 */
        *is_find = 1;
        *is_root = 1;
        dentry_ret = super.root_dentry;
    }
}

```

如果路径层级大于 0，该函数使用 `strtok` 分割路径，根据每个路径组件逐级查找对应的目录项。在查找过程中，若遇到缓存中未加载的 `inode`，会调用 `newfs_read_inode` 进行加载：

```

name = strtok(path_cpy, "/");
while (name)
{
    lvl++;
    if (dentry_cursor->inode == NULL) { /* Cache机制 */
        newfs_read_inode(dentry_cursor, dentry_cursor->ino);
    }
}

```

接下来，该函数会遍历每一级目录的子目录项，直到找到匹配的目录项。如果在某一级未找到匹配的子目录项，则标记为未找到并退出。若找到目标目录项，函数会返回该目录项并更新 `is_find` 为 1。如果路径中的某个部分是文件而非目录，函数会提前退出并返回错误。最终，`newfs_lookup` 返回找到的目录项（`dentry_ret`），并且如果该目录项的 `inode` 尚未加载，函数会进行加载：

```

inode = dentry_cursor->inode;

if (inode->dentry->ftype == NEWFS_TYPE_FILE && lvl < total_lvl) {
    dentry_ret = inode->dentry;
    break;
}

if (inode->dentry->ftype == NEWFS_TYPE_DIR) {
    dentry_cursor = inode->dentry;
    is_hit = 0;

    while (dentry_cursor) /* 遍历子目录项 */
    {
        if (memcmp(dentry_cursor->name, name, strlen(name)) == 0) {
            is_hit = 1;
            break;
        }
        dentry_cursor = dentry_cursor->brother;
    }

    if (!is_hit) {
        *is_find = 0;
        dentry_ret = inode->dentry;
        break;
    }

    if (is_hit && lvl == total_lvl) {
        *is_find = 1;
        dentry_ret = dentry_cursor;
        break;
    }
}

name = strtok(NULL, "/");
}

if (dentry_ret->inode == NULL) {
    dentry_ret->inode = newfs_read_inode(dentry_ret, dentry_ret->ino);
}

return dentry_ret;

```

其中，`newfs_alloc_dentry` 函数用于为给定的 `inode` 分配一个新的目录项。首先，函数检查 `inode` 的 `dir_cnt` 是否为 0，如果是，则意味着该 `inode` 尚未分配任何数据块，则该函数遍历数据位图，查找第一个空闲的数据块，并将该块标记为已使用，分配给新目录

项，如果该 inode 已有目录项，则直接为目录项分配第一个已有的数据位图标记：

```
int newfs_alloc_dentry(struct newfs_inode* inode, struct newfs_dentry* dentry) {
    int byte_cursor = 0;
    int bit_cursor = 0;
    int dno_cursor = 0;
    int is_find_free = 0;

    if (inode->dir_cnt == 0) {
        for (byte_cursor = 0; byte_cursor < super.data_map_blks * super.sz_blks; byte_cursor++) {
            for (bit_cursor = 0; bit_cursor < 8; bit_cursor++) {
                if((super.data_map[byte_cursor] & (0x1 << bit_cursor)) == 0) {
                    super.data_map[byte_cursor] |= (0x1 << bit_cursor);
                    is_find_free = 1;
                    break;
                }
                dno_cursor++;
            }
            if (is_find_free) {
                break;
            }
        }
        dentry->dno = dno_cursor;
    } else {
        dentry->dno = inode->dentries->dno;
    }
    inode->dno = dentry->dno;
}
```

接着，函数将目录项添加到 inode 的 dentries 链表中，若该 inode 的 dentries 为 NULL，则直接将新目录项作为首个子目录项。最后，更新 inode 的 dir_cnt 并返回目录项数量：

```
if (inode->dentries == NULL) {
    inode->dentries = dentry;
} else {
    dentry->brother = inode->dentries;
    inode->dentries = dentry;
}
inode->dir_cnt++;
return inode->dir_cnt;
```

4. 创建文件

```
int newfs_mknod(const char* path, mode_t mode, dev_t dev) {
    /* TODO: 解析路径，并创建相应的文件 */
    int is_find, is_root;

    struct newfs_dentry* last_dentry = newfs_lookup(path, &is_find, &is_root);
    struct newfs_dentry* dentry;
    struct newfs_inode* inode;
    char* name;

    if (is_find == 1) {
        return -17;
    }

    name = newfs_get_fname(path);

    if (S_ISREG(mode)) {
        dentry = new_dentry(name, NEWFS_TYPE_FILE);
    }
    else if (S_ISDIR(mode)) {
        dentry = new_dentry(name, NEWFS_TYPE_DIR);
    }
    else {
        dentry = new_dentry(name, NEWFS_TYPE_FILE);
    }
    dentry->parent = last_dentry;
    inode = newfs_alloc_inode(dentry);
    newfs_alloc_dentry(last_dentry->inode, dentry);
    return 0;
}
```

newfs_mknod 函数使用 newfs_lookup 查找给定路径的最后一个目录项，并检查该路径是否已存在。如果路径已存在，返回错误码：

```
int newfs_mknod(const char* path, mode_t mode, dev_t dev) {
    /* TODO: 解析路径, 并创建相应的文件 */
    int is_find, is_root;

    struct newfs_dentry* last_dentry = newfs_lookup(path, &is_find, &is_root);
    struct newfs_dentry* dentry;
    struct newfs_inode* inode;
    char* name;

    if (is_find == 1) {
        return -17;
    }
}
```

然后, 函数提取路径中的文件名, 并根据给定的模式 (mode) 判断要创建的是文件还是目录:

```
if (S_ISREG(mode)) {
    dentry = new_dentry(name, NEWFS_TYPE_FILE);
}
else if (S_ISDIR(mode)) {
    dentry = new_dentry(name, NEWFS_TYPE_DIR);
}
else {
    dentry = new_dentry(name, NEWFS_TYPE_FILE);
}
```

最后, 将新创建的目录项与父目录关联, 并为其分配一个 inode, 再调用 newfs_alloc_dentry 将新目录项添加到父目录的目录项列表中:

```
dentry->parent = last_dentry;
inode = newfs_alloc_inode(dentry);
newfs_alloc_dentry(last_dentry->inode, dentry);
return 0;
```

5. 显示文件

newfs_getattr 函数首先调用 newfs_lookup 查找路径对应的目录项。如果路径未找到, 返回错误码:

```
int newfs_getattr(const char* path, struct stat * newfs_stat) {
    /* TODO: 解析路径, 获取inode, 填充newfs_stat, 可参考/fs/simplefs/sfs.c的sfs_getattr()函数实现 */
    int is_find, is_root;
    struct newfs_dentry* dentry = newfs_lookup(path, &is_find, &is_root);
    if (is_find == 0) {
        return -2;
    }
}
```

然后, 根据目录项类型 (文件或目录) 来设置 newfs_stat 的属性

```
if (dentry->inode->dentry->ftype == NEWFS_TYPE_DIR) {
    newfs_stat->st_mode = S_IFDIR | NEWFS_DEFAULT_PERM;
    newfs_stat->st_size = dentry->inode->dir_cnt * sizeof(struct newfs_dentry_d);
}
else if (dentry->inode->dentry->ftype == NEWFS_TYPE_FILE) {
    newfs_stat->st_mode = S_IFREG | NEWFS_DEFAULT_PERM;
    newfs_stat->st_size = dentry->inode->size;
}
```

接着, 设置其他属性, 包括链接数、文件所有者的用户 ID 和组 ID、访问时间和修改时间, 以及块大小等:

```
newfs_stat->st_nlink = 1;
newfs_stat->st_uid = getuid();
newfs_stat->st_gid = getgid();
newfs_stat->st_atime = time(NULL);
newfs_stat->st_mtime = time(NULL);
newfs_stat->st_blksize = super.sz_blks;

if (is_root) {
    newfs_stat->st_size = super.sz_usage;
    newfs_stat->st_blocks = super.blks_nums;
    newfs_stat->st_nlink = 2; /* !特殊, 根目录link数为2 */
}

return 0;
```

6. 读取目录

newfs_readdir 函数首先使用 newfs_lookup 查找给定路径对应的目录项, 并获取该目录

项的 inode。如果路径被找到，函数通过 `newfs_get_dentry` 获取指定偏移量下的子目录项，再使用 FUSE 提供的 `filler` 函数将子目录项的名称添加到 `buf` 中，填充目录的内容。为了支持目录遍历，`offset` 会在每次填充时递增：

```
int newfs_readdir(const char * path, void * buf, fuse_fill_dir_t filler, off_t offset,
                  struct fuse_file_info * fi) {
    /* TODO: 解析路径，获取目录的Inode，并读取目录项，利用filler填充到buf，可参考/fs/simple
    int is_find, is_root;
    int cur_dir = offset;

    struct newfs_dentry* dentry = newfs_lookup(path, &is_find, &is_root);
    struct newfs_dentry* sub_dentry;
    struct newfs_inode* inode;
    if (is_find) {
        inode = dentry->inode;
        sub_dentry = newfs_get_dentry(inode, cur_dir);
        if (sub_dentry) {
            filler(buf, sub_dentry->name, NULL, ++offset);
        }
        return 0;
    }
    return -2;
}
```

3、 实验特色

实验中你认为自己实现的比较有特色的部分，包括设计思路、实现方法和预期效果。

1. 实现了文件的灵活读写，即调用读写文件的函数前无需先移动磁盘头对齐和读写大小可以不固定。
2. 采用递归的方式在重新挂载时读取文件的层级结构。
3. 在申请时按需分配索引节点和数据块。

二、遇到的问题及解决方法

列出实验过程中遇到的主要问题，包括技术难题、设计挑战等。对应每个问题，提供采取的解决策略，以及解决问题后的效果评估。

1. 问题：不了解怎么对文件系统进行操作，怎么设计文件系统。

解决方法：仔细研究 `simplefs` 的代码，熟悉文件系统功能实现的主要操作的细节。

2. 问题：数据区写回位置错误。

解决方法：断点调试，逐个文件检查其写回位置和其在数据位图被分配的位置是否一致，最终发现是挂载时数据区偏移设置有误。

3. 问题：重新挂载后无法读取到文件。

解决方法：断点调试，检查重新挂载后文件层次结构建立的过程，发现递归建立结构时数据索引计算出错，更改递归逻辑和索引计算得以解决。

三、实验收获和建议

实验中的收获、感受、问题、建议等。

简易的 ext2 文件系统实验让我对操作系统中文件系统的核心原理和实现细节有了更深入的理解。通过实现 inode 结构、目录项、数据块分配等功能，我对磁盘上的文件是如何被组织、存取、修改等有了更清晰的认识。特别是在 inode 的设计上，我意识到它是文件系统中至关重要的组件，它不仅存储文件的元数据，还通过指针链接到数据块，管理文件的内容和位置。在实现目录项和目录结构时，我体会到了如何高效地组织文件和文件夹结构，以便快速地检索和定位文件。同时，位图的设计和实现让我更加理解了磁盘块的分配与回收，如何避免碎片化并最大化磁盘利用率。

目前整体实验的测试用例还是比较少，后续可以在测试中添加建立非空文件，增加更多的重新挂载测试用例。

四、参考资料

实验过程中查找的信息和资料

《simplefs 文件系统参考实现》<https://os-labs.pages.dev/lab5/part6/>